

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning Libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
In [2]: cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)

df['target'] = cancer.target

df.head()
```

```
Out[2]:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809

5 rows × 31 columns

```
In [3]: print("Dataset Shape:")
print(df.shape)
```

Dataset Shape:
(569, 31)

```
In [5]: print(df.columns)
print(df.info())
print(df.describe())
```

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
      'mean smoothness', 'mean compactness', 'mean concavity',
      'mean concave points', 'mean symmetry', 'mean fractal dimension',
      'radius error', 'texture error', 'perimeter error', 'area error',
      'smoothness error', 'compactness error', 'concavity error',
      'concave points error', 'symmetry error', 'fractal dimension error',
      'worst radius', 'worst texture', 'worst perimeter', 'worst area',
      'worst smoothness', 'worst compactness', 'worst concavity',
      'worst concave points', 'worst symmetry', 'worst fractal dimension',
      'target'],
      dtype='object')
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 569 entries, 0 to 568
```

```
Data columns (total 31 columns):
```

#	Column	Non-Null Count	Dtype
0	mean radius	569 non-null	float64
1	mean texture	569 non-null	float64
2	mean perimeter	569 non-null	float64
3	mean area	569 non-null	float64
4	mean smoothness	569 non-null	float64
5	mean compactness	569 non-null	float64
6	mean concavity	569 non-null	float64
7	mean concave points	569 non-null	float64
8	mean symmetry	569 non-null	float64
9	mean fractal dimension	569 non-null	float64
10	radius error	569 non-null	float64
11	texture error	569 non-null	float64
12	perimeter error	569 non-null	float64
13	area error	569 non-null	float64
14	smoothness error	569 non-null	float64
15	compactness error	569 non-null	float64
16	concavity error	569 non-null	float64
17	concave points error	569 non-null	float64
18	symmetry error	569 non-null	float64
19	fractal dimension error	569 non-null	float64
20	worst radius	569 non-null	float64
21	worst texture	569 non-null	float64
22	worst perimeter	569 non-null	float64
23	worst area	569 non-null	float64
24	worst smoothness	569 non-null	float64
25	worst compactness	569 non-null	float64
26	worst concavity	569 non-null	float64
27	worst concave points	569 non-null	float64
28	worst symmetry	569 non-null	float64
29	worst fractal dimension	569 non-null	float64
30	target	569 non-null	int32

```
dtypes: float64(30), int32(1)
```

```
memory usage: 135.7 KB
```

```
None
```

	mean radius	mean texture	mean perimeter	mean area \
count	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104
std	3.524049	4.301036	24.298981	351.914129
min	6.981000	9.710000	43.790000	143.500000
25%	11.700000	16.170000	75.170000	420.300000
50%	13.370000	18.840000	86.240000	551.100000
75%	15.780000	21.800000	104.100000	782.700000
max	28.110000	39.280000	188.500000	2501.000000

	mean smoothness	mean compactness	mean concavity	mean concave points \
count	569.000000	569.000000	569.000000	569.000000
mean	0.096360	0.104341	0.088799	0.048919
std	0.014064	0.052813	0.079720	0.038803

min	0.052630	0.019380	0.000000	0.000000
25%	0.086370	0.064920	0.029560	0.020310
50%	0.095870	0.092630	0.061540	0.033500
75%	0.105300	0.130400	0.130700	0.074000
max	0.163400	0.345400	0.426800	0.201200

	mean symmetry	mean fractal dimension	... worst texture \
count	569.000000	569.000000	569.000000
mean	0.181162	0.062798	25.677223
std	0.027414	0.007060	6.146258
min	0.106000	0.049960	12.020000
25%	0.161900	0.057700	21.080000
50%	0.179200	0.061540	25.410000
75%	0.195700	0.066120	29.720000
max	0.304000	0.097440	49.540000

	worst perimeter	worst area	worst smoothness	worst compactness \
count	569.000000	569.000000	569.000000	569.000000
mean	107.261213	880.583128	0.132369	0.254265
std	33.602542	569.356993	0.022832	0.157336
min	50.410000	185.200000	0.071170	0.027290
25%	84.110000	515.300000	0.116600	0.147200
50%	97.660000	686.500000	0.131300	0.211900
75%	125.400000	1084.000000	0.146000	0.339100
max	251.200000	4254.000000	0.222600	1.058000

	worst concavity	worst concave points	worst symmetry \
count	569.000000	569.000000	569.000000
mean	0.272188	0.114606	0.290076
std	0.208624	0.065732	0.061867
min	0.000000	0.000000	0.156500
25%	0.114500	0.064930	0.250400
50%	0.226700	0.099930	0.282200
75%	0.382900	0.161400	0.317900
max	1.252000	0.291000	0.663800

	worst fractal dimension	target
count	569.000000	569.000000
mean	0.083946	0.627417
std	0.018061	0.483918
min	0.055040	0.000000
25%	0.071460	0.000000
50%	0.080040	1.000000
75%	0.092080	1.000000
max	0.207500	1.000000

[8 rows x 31 columns]

```
In [6]: print(df.isnull().sum())
```

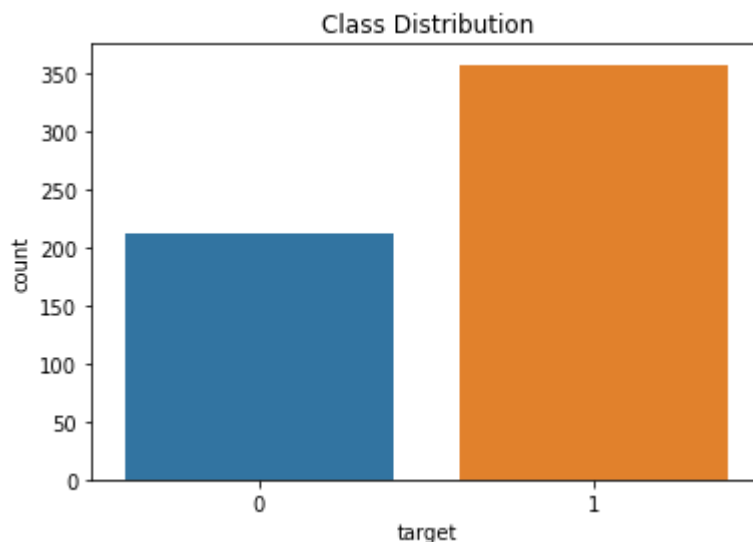
```
mean radius      0
mean texture     0
mean perimeter   0
mean area        0
mean smoothness  0
mean compactness 0
mean concavity   0
mean concave points 0
mean symmetry    0
mean fractal dimension 0
radius error     0
texture error    0
perimeter error  0
area error       0
smoothness error 0
compactness error 0
concavity error  0
concave points error 0
symmetry error   0
fractal dimension error 0
worst radius     0
worst texture    0
worst perimeter  0
worst area       0
worst smoothness 0
worst compactness 0
worst concavity  0
worst concave points 0
worst symmetry   0
worst fractal dimension 0
target          0
dtype: int64
```

```
In [7]: print(df['target'].value_counts())
```

```
1    357
0    212
Name: target, dtype: int64
```

Benign: 62.7% Malignant: 37.3% This is only a moderate imbalance

```
In [8]: sns.countplot(x='target', data=df)
plt.title("Class Distribution")
plt.show()
```



```
In [9]: X = df.drop('target', axis=1)
        y = df['target']

        X_train, X_test, y_train, y_test = train_test_split(
            X,
            y,
            test_size=0.2,
            random_state=42
        )

        print(X_train.shape)
        print(X_test.shape)

(455, 30)
(114, 30)
```

```
In [10]: scaler = StandardScaler()

        X_train_scaled = scaler.fit_transform(X_train)

        X_test_scaled = scaler.transform(X_test)
```

Using Logistic Regression

```
In [13]: log_model = LogisticRegression(random_state=42)

        log_model.fit(X_train_scaled, y_train)

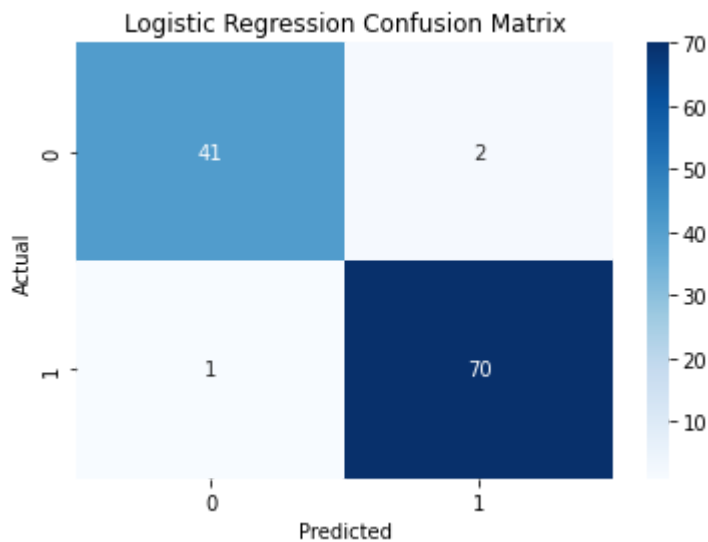
        y_pred_log = log_model.predict(X_test_scaled)
```

```
In [14]: accuracy_log = accuracy_score(y_test, y_pred_log)
        print("Accuracy:", accuracy_log)
        precision_log = precision_score(y_test, y_pred_log)
        print("Precision:", precision_log)
        recall_log = recall_score(y_test, y_pred_log)
        print("Recall:", recall_log)
        f1_log = f1_score(y_test, y_pred_log)
        print("F1 Score:", f1_log)
        cm_log = confusion_matrix(y_test, y_pred_log)

        sns.heatmap(cm_log,
                     annot=True,
                     fmt='d',
                     cmap='Blues')

        plt.xlabel("Predicted")
        plt.ylabel("Actual")
        plt.title("Logistic Regression Confusion Matrix")
        plt.show()
```

Accuracy: 0.9736842105263158
Precision: 0.9722222222222222
Recall: 0.9859154929577465
F1 Score: 0.979020979020979



The Logistic Regression model achieved an accuracy of 97.37%, with 97.22% precision, 98.59% recall, and an F1-score of 97.90%. These results indicate excellent classification performance and a high ability to correctly identify both benign and malignant tumors.

```
In [15]: print(classification_report(y_test, y_pred_log))
```

	precision	recall	f1-score	support
0	0.98	0.95	0.96	43
1	0.97	0.99	0.98	71
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

The classification report indicates consistently high precision, recall, and F1-scores for both classes, confirming that the Logistic Regression model performs reliably across benign and malignant cases.

```
In [25]: sample = X_test_scaled[0].reshape(1, -1)

# Compute z = w·x + b (decision function)
z = log_model.decision_function(sample)

# Compute probability using sigmoid
prob = 1 / (1 + np.exp(-z))

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Generate z values
z_values = np.linspace(-6, 6, 100)

# Calculate sigmoid values
sigmoid_probs = sigmoid(z_values)

# Plot
plt.figure(figsize=(10,6))
plt.plot(z_values, sigmoid_probs, linewidth=2, label='Sigmoid Function')

# Highlight example point
plt.scatter(z, prob,
```

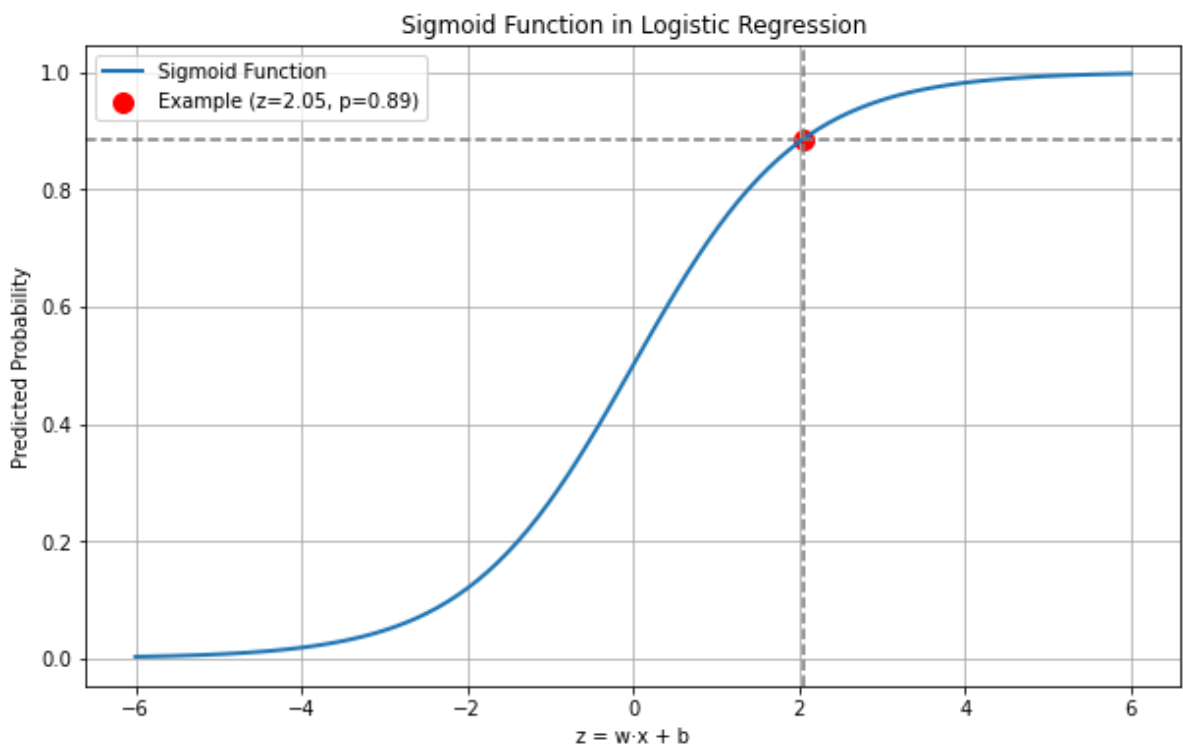
```

        color='red',
        s=100,
        label=f'Example (z={z[0]:.2f}, p={prob[0]:.2f})')

plt.axvline(z[0], color='gray', linestyle='--')
plt.axhline(prob[0], color='gray', linestyle='--')

plt.xlabel("z = w·x + b")
plt.ylabel("Predicted Probability")
plt.title("Sigmoid Function in Logistic Regression")
plt.grid(True)
plt.legend()
plt.show()

```



The sigmoid function converts the linear output of Logistic Regression into a probability between 0 and 1, enabling binary classification using a decision threshold.

```

In [26]: # Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Generate z values
z_values = np.linspace(-8, 8, 200)

# Calculate probabilities
probabilities = sigmoid(z_values)

plt.figure(figsize=(10,6))

# Sigmoid curve
plt.plot(z_values, probabilities,
         color='blue',
         linewidth=3,
         label="Sigmoid Function")

# Decision threshold

```

```

plt.axhline(y=0.5,
            color='red',
            linestyle='--',
            label="Decision Boundary (0.5)")

# High confidence zones
plt.axvspan(-8, -2.5,
            color='purple',
            alpha=0.15)

plt.axvspan(2.5, 8,
            color='green',
            alpha=0.15)

# Maximum confusion point
plt.scatter(0,0.5,
            color='red',
            s=100,
            zorder=5)

plt.annotate(
    "Maximum Confusion\nProbability ≈ 0.5",
    xy=(0,0.5),
    xytext=(1.5,0.35),
    arrowprops=dict(
        arrowstyle="→",
        color="red"
    )
)

# High confidence labels

plt.annotate(
    "High Confidence\nClass 0\n( $p \rightarrow 0$ )",
    xy=(-5, sigmoid(-5)),
    xytext=(-7,0.25),
    arrowprops=dict(
        arrowstyle="→",
        color="purple"
    ),
    color="purple"
)

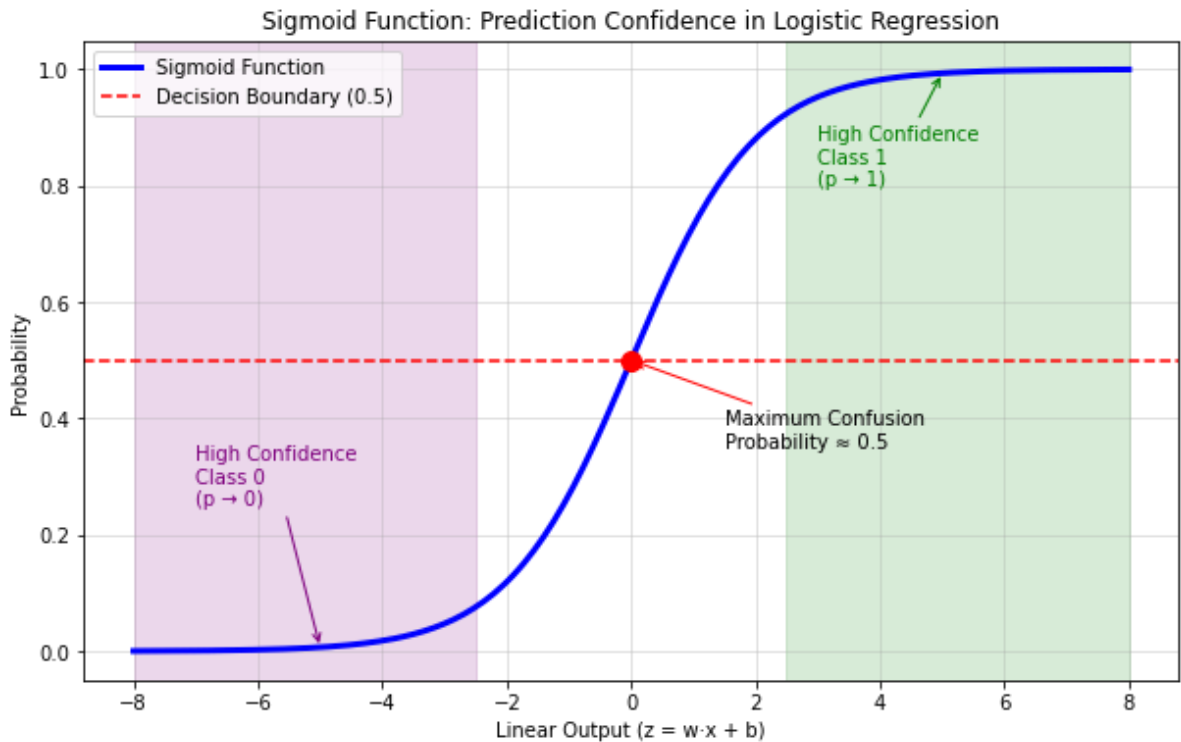
plt.annotate(
    "High Confidence\nClass 1\n( $p \rightarrow 1$ )",
    xy=(5, sigmoid(5)),
    xytext=(3,0.8),
    arrowprops=dict(
        arrowstyle="→",
        color="green"
    ),
    color="green"
)

plt.xlabel("Linear Output ( $z = w \cdot x + b$ )")
plt.ylabel("Probability")
plt.title("Sigmoid Function: Prediction Confidence in Logistic Regression")

plt.grid(alpha=0.4)

```

```
plt.legend()
plt.show()
```



Predictions with probabilities close to 0 or 1 represent high-confidence classifications. The region around probability 0.5 represents maximum uncertainty because both classes have an equal likelihood of being predicted. Samples near this decision boundary/threshold have a higher chance of misclassification.

```
In [ ]: # probabilities from the trained model
probs = log_model.predict_proba(X_test_scaled)[: ,1]

# most uncertain sample
uncertain_index = np.argmin(abs(probs - 0.5))

print("Most uncertain sample probability:",
      probs[uncertain_index])
```

Most uncertain sample probability: 0.5132881101835551

```
In [ ]: scores = []

for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)
    scores.append(accuracy_score(y_test, y_pred))

best_k = scores.index(max(scores)) + 1
print("Best k:", best_k)
```

Best k: 9

The optimal value of $k = 9$ was selected based on model performance.

```
In [18]: knn = KNeighborsClassifier(n_neighbors=9)

knn.fit(X_train_scaled, y_train)
```

```
y_pred_knn = knn.predict(X_test_scaled)
```

```
In [19]: accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("Accuracy:", accuracy_knn)
precision_knn = precision_score(y_test, y_pred_knn)
print("Precision:", precision_knn)
recall_knn = recall_score(y_test, y_pred_knn)
print("Recall:", recall_knn)
f1_knn = f1_score(y_test, y_pred_knn)
print("F1 Score:", f1_knn)
cm_knn = confusion_matrix(y_test, y_pred_knn)

sns.heatmap(cm_knn,
            annot=True,
            fmt='d',
            cmap='Greens')

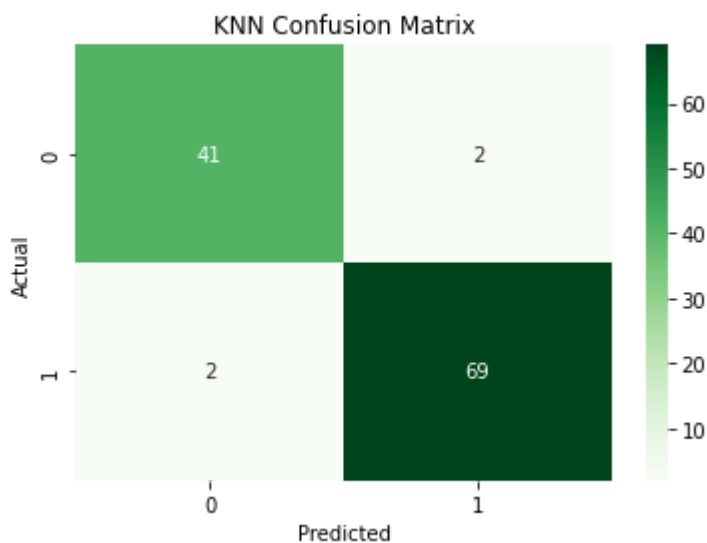
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")
plt.show()
```

Accuracy: 0.9649122807017544

Precision: 0.971830985915493

Recall: 0.971830985915493

F1 Score: 0.971830985915493



The KNN classifier achieved an accuracy of 96.49%, with 97.18% precision, 97.18% recall, and an F1-score of 97.18%. The model demonstrated strong classification performance with only a slight reduction in accuracy compared to Logistic Regression.

```
In [20]: print(classification_report(y_test, y_pred_knn))
```

	precision	recall	f1-score	support
0	0.95	0.95	0.95	43
1	0.97	0.97	0.97	71
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114

The classification report shows balanced precision, recall, and F1-scores for both classes, indicating that the KNN model performs consistently on the test data.

```
In [21]: comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "KNN"],
    "Accuracy": [accuracy_log, accuracy_knn],
    "Precision": [precision_log, precision_knn],
    "Recall": [recall_log, recall_knn],
    "F1 Score": [f1_log, f1_knn]
})

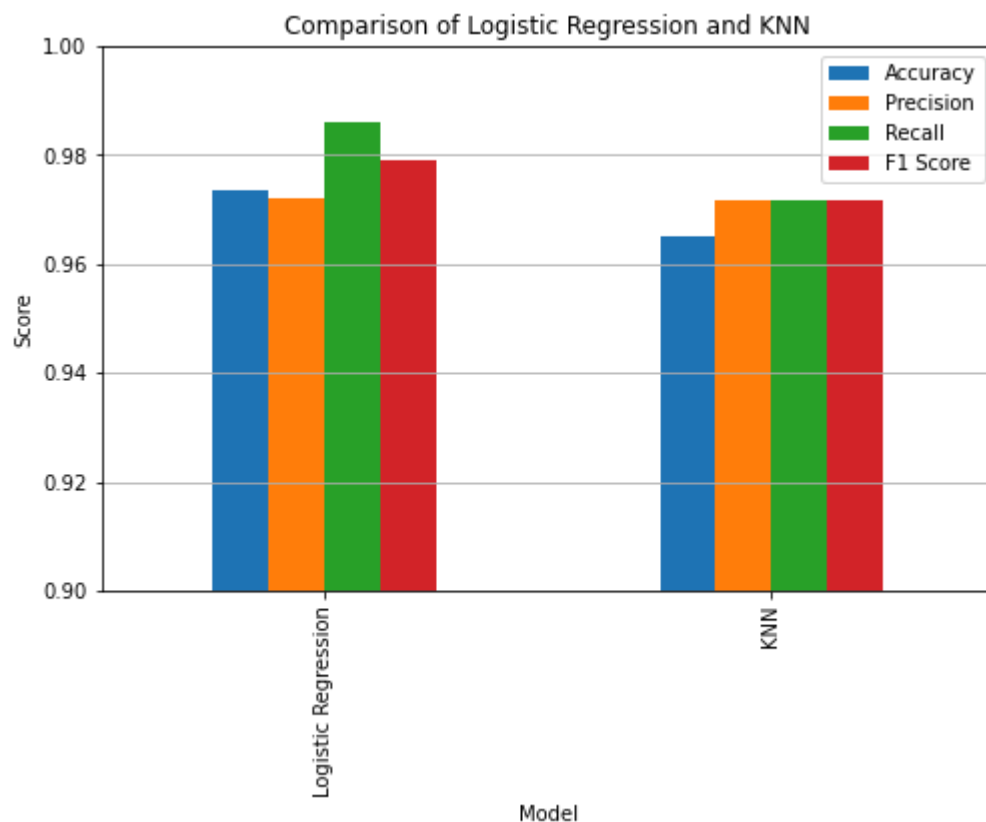
comparison
```

```
Out[21]:
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.973684	0.972222	0.985915	0.979021
1	KNN	0.964912	0.971831	0.971831	0.971831

```
In [22]: comparison.set_index("Model").plot(
    kind="bar",
    figsize=(8,5)
)

plt.title("Comparison of Logistic Regression and KNN")
plt.ylabel("Score")
plt.ylim(0.9, 1.0)
plt.grid(axis='y')
plt.show()
```



```
In [24]: if accuracy_log > accuracy_knn:
    print("Logistic Regression performed better.")
elif accuracy_knn > accuracy_log:
    print("KNN performed better.")
else:
```

```
print("Both models performed equally.")  
  
print("Difference between both models: ", accuracy_log - accuracy_knn)
```

Logistic Regression performed better.

Difference between both models: 0.00877192982456143

Logistic Regression achieved a slightly higher accuracy (97.37%) than KNN (96.49%), along with higher recall (98.59%) and F1-score (97.90%).